МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ» (ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики

Кафедра математического обеспечения ЭВМ

Цифровая обработка изображений

Лабораторная работа № 2

Студент: Ф. И. О.

Преподаватель: Сырых А. С.

Воронеж 2026

Содержание

1. Постановка задачи..... 3
2. Математическое описание алгоритма..... 4
2.1. Алгоритмы выделения контуров..... 4
2.2. Линейные усредняющие фильтры..... 8
3. Программное описание алгоритма..... 10
4. Результат работы программы..... 11
5. Заключение..... 15
Приложение А..... 16
Приложение Б..... 19

1. Постановка задачи

1) Вариант № 1. Выделение контуров на изображении:
    a) оператор Лапласа;
    b) оператор Робертса;
    c) оператор Собела.
2) Вариант № 2. Удаление гауссова шума с изображений в пространственной области усредняющими фильтрами. Шум добавляется программно с фиксированным начальным состоянием генератора.

2. Математическое описание алгоритма

Изображение рассматривается как двумерная дискретная функция $I=(x,y)$, где $x \in [0, W-1]$, $y \in [0, H-1]$, W и H — соответственно ширина и высота растра в пикселях, а значение $I=(x,y) \in [0,255]$ соответствует яркости в узле сетки. Для цветных изображений предварительно осуществляется переход к полутоновому представлению (grayscale) по стандартной психофизической формуле яркости (1):

$$Y(x,y)=0.299*R(x,y)+0.587*G(x,y)+0.114*B(x,y) \qquad (1)$$

2.1. Алгоритмы выделения контуров

Контуры характеризуются резкими локальными перепадами яркости, которые определяются производными первого и второго порядков.

Оператор Собела

Осуществляет вычисление градиента функции яркости с предварительным локальным сглаживанием, перпендикулярным направлению дифференцирования. Применяются свертки с двумя пространственными масками размера 3×3 (2):

$$G_x=\begin{pmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{pmatrix}, G_y=\begin{pmatrix}-1 & -2 & -1\\ 0 & 0 & 0\\ 1 & 2 & 1\end{pmatrix} \quad (2)$$

Итоговая амплитуда градиента рассчитывается по евклидовой норме (3) с отсечением диапазона(4):

$$G(x,y)=\sqrt{G_x^2(x,y)+G_y^2(x,y)} \quad (3)$$

$$I_{out}(x,y)=min\left(255,max\left(0,[G(x,y)]\right)\right) \quad (4)$$

Алгоритм 1. Оператор Собела

1) Преобразовать входное изображение в полутоновое по формуле (1).
2) Для каждого внутреннего пикселя (x,y) вычислить свертки $G_x(x,y)$ и $G_y(x,y)$ по маскам (2).
3) Найти модуль вектора градиента по формуле (3).
4) Нормализовать полученное значение в диапазон $[0,255]$ по формуле (4).

Оператор Робертса

Использует диагональные конечные разности на компактной апертуре размера 2×2 (5):

$$G_x=\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}, G_y=\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix} \quad (5)$$

Аналитическое вычисление компонент разностей:

$$G_x(x,y)=I(x,y)-I(x+1,y+1), G_y(x,y)=I(x+1,y)-I(x,y+1) \quad (6)$$

Значение амплитуды вычисляется по формуле (3) и ограничивается формулой (4).

Алгоритм 2. Оператор Робертса

1) Получить яркостную матрицу $I(x,y)$ по формуле (1).
2) Для каждой точки (x,y) вычислить диагональные разности согласно соотношениям (6).
3) Определить амплитуду градиента по формуле (3).
4) Записать результат в выходное изображение с учетом нормализации (4).

Оператор Лапласа

Представляет собой изотропный оператор второй пространственной производной (7):

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \quad (7)$$

В дискретном двумерном пространстве оператор задается ядром свертки 3×3 (8):

$$L = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad (8)$$

Отклик оператора берется по абсолютному значению для формирования границ (9):

$$I_{out}(x, y) = min\left(255, \left|(L * I)(x, y)\right|\right) \quad (9)$$

Алгоритм 3. Оператор Лапласа

1) Сформировать полутоновое изображение по формуле (1).
2) Выполнить двумерную дискретную свертку матрицы изображения с ядром L (8).
3) Взять абсолютное значение полученного результата согласно (9).

2.2.Линейные усредняющие фильтры

Усредняющие фильтры предназначены для подавления аддитивного гауссова шума за счет взвешенного интегрирования значений в окрестности точки.

Прямоугольный усредняющий фильтр (Box Blur)

Значение пикселя заменяется средним арифметическим значений в окрестности размера $k \times k$ с радиусом $r = \lfloor k/2 \rfloor$ (10):

$$I_{out}(x, y) = \frac{1}{k^2} \sum_{i=-r}^{r} \sum_{j=-r}^{r} I(x+i, y+j) \qquad (10)$$

Алгоритм 4. Прямоугольное усреднение (Box Blur)

1) Задать нечетный линейный размер ядра k и вычислить радиус $r = \lfloor k/2 \rfloor$.
2) Для каждого пикселя (x, y) просуммировать яркости всех k^2 соседей.
3) Разделить сумму на k^2 согласно формуле (10) и записать в результирующий массив.

Фильтр Гаусса (Gaussian Blur)

Весовые коэффициенты определяются непрерывным двумерным распределением Гаусса (11):

$$G(x,y)=\frac{1}{2\pi\sigma^2}e^{\frac{-(x^2+y^2)}{2\sigma^2}} \qquad (11)$$

где σ – среднеквадратическое отклонение, определяющее радиус сглаживания. Дискретная матрица W размера $k\times k$ с индексами $i,j\in[-r,r]$ подлежит строгой нормировке (12):

$$w(i,j)=\frac{G(i,j)}{\sum_{u=-r}^{r}\sum_{v=-r}^{r}G(u,v)} \qquad (12)$$

Результирующий сигнал рассчитывается по формуле взвешенной свертки (13):

$$I_{out}(x,y)=\sum_{i=-r}^{r}\sum_{j=-r}^{r}w(i,j)*I(x+i,y+j) \qquad (13)$$

Алгоритм 5. Фильтр Гаусса

1) Задать размер ядра k и параметр σ.
2) Сформировать матрицу весов с использованием формулы (11) и нормировать ее элементы по формуле (12).
3) Применить ядро фильтрации к каждому пикселю изображения в соответствии с формулой (13).

3. Программное описание алгоритма

Программная реализация выполнена на языке C++17. Класс Image реализует загрузку и сохранение PNG, работу с 1–4 каналами и глубокое копирование. Новые методы в его интерфейс не добавлялись. Алгоритмы обработки находятся в пространстве имен Filters.

Функция Filters::applySobel принимает на вход объект const Image&. В своей реализации она переводит изображение в полутоновое по формуле (1), проходит по каждому пикселю и применяет алгоритм 1, вычисляя свертки по маскам (2) и результирующий градиент по формуле (3) с ограничением (4). Реализация представлена в Приложении А.

Функция Filters::applyRoberts принимает на вход объект const Image&. Она реализует алгоритм 2, вычисляя разности по формуле (6) для масок (5) и определяя амплитуду градиента по формуле (3). Ее реализация представлена в Приложении А.

Функция Filters::applyLaplacian принимает на вход полутоновое изображение и выполняет алгоритм 3, используя дискретную маску Лапласа (8) и взятие модуля по формуле (9). Ее реализация представлена в Приложении А.

Функция Filters::applyBoxBlur принимает на вход исходное изображение и размер ядра kernelSize. Она использует алгоритм 4, суммируя значения пикселей в окне и деля результат на площадь окна согласно формуле (10). Ее реализация представлена в Приложении Б.

Функция Filters::applyGaussianBlur принимает размер матрицы kernelSize и параметр sigma. В своей реализации она генерирует двумерное ядро по формуле (11), нормализует его по формуле (12) и выполняет свертку по формуле (13), реализуя алгоритм 5. Реализация представлена в Приложении Б.

4. Результат работы программы

Для проверки программа самостоятельно создаёт полутоновое изображение 256×256: квадрат яркости 200 на фоне 50, круг яркости 150, градиент и тонкие полосы. Затем добавляется гауссов шум $N(0, 25^2)$; используется mt19937 с seed=42. Чистый эталон сохраняется в test_input/clean.png, зашумлённый вход — в test_input/input.png. Изображения из архивов не используются.

Множество мелких деталей и четких границ — это позволяет проверить качество алгоритмов при выделении контуров;

Равномерные участки большой площади — для оценки эффективности подавления аддитивного шума на областях без резких изменений яркости;

Градиент в нижней части — для выявления потери плавных переходов; вертикальные полосы шириной 3 пикселя — для сравнения сохранности мелких деталей после усреднения.

Преимущества и недостатки алгоритмов рассматриваются в сравнении с математической моделью:

При преобразовании результатов в целые числа диапазон ограничивается [0, 255]. Для Box Blur используется целочисленное деление, для Gaussian Blur — округление к ближайшему целому. На краях координаты ограничиваются ближайшим допустимым пикселем (репликация границ).

На чистом изображении (дополнительные файлы clean_sobel.png, clean_roberts.png, clean_laplacian.png) операторы выделяют границы квадрата, круга и полос. Собел использует усреднение в поперечном направлении, но его ненормированная амплитуда также усиливает шум и может достигать ограничения 255.

Оператор Робертса формирует наиболее тонкие границы объектов благодаря окну 2×2, однако из-за отсутствия сглаживания он крайне чувствителен к гауссову шуму, что привело к появлению хаотичных ложных контуров на однородном фоне.

Лапласиан использует вторые разности: на шумном фоне заметны многочисленные ложные контуры. На резкой границе отклик возникает с двух сторон перепада; после взятия модуля знак второй производной теряется.

Прямоугольный фильтр 5×5 сильнее снижает ошибку на однородном фоне, чем окно 3×3, но сильнее размывает края и тонкие полосы. Гауссов фильтр 5×5 с σ=1,5 даёт больший вес центру окна; его результат представляет компромисс между сглаживанием и сохранением переходов.

Результаты измерений относительно чистого оригинала (MSE; PSNR, дБ; MSE на однородном фоне): шум — 602.20; 20.33; 613.45; окно 3×3 — 232.66; 24.46; 67.87; окно 5×5 — 355.90; 22.62; 23.54; Гаусс 5×5, σ=1,5 — 252.24; 24.11; 30.32.

$MSE = \Sigma(I - I_0)^2/(WH)$; $PSNR = 10 \lg(255^2/MSE)$. Область однородного фона: $5 \le x < 45$, $100 \le y < 200$. На данном тесте наименьшую общую MSE даёт окно 3×3: выигрыш окна 5×5 на плоском фоне компенсируется ошибкой на границах. Ниже приведены зашумлённый вход и результаты первых двух вариантов.

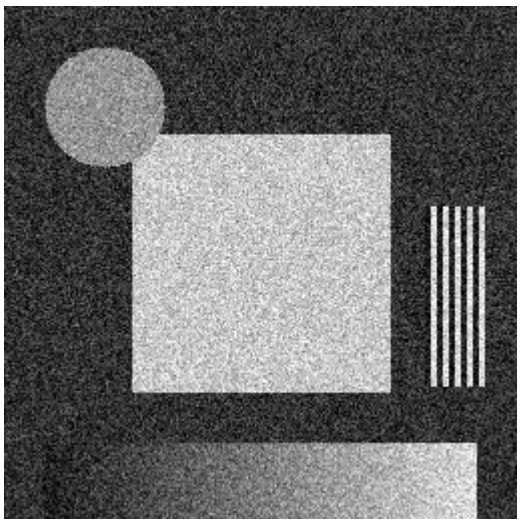


Рис. 1. Программно сгенерированное изображение с гауссовым шумом, σ = 25

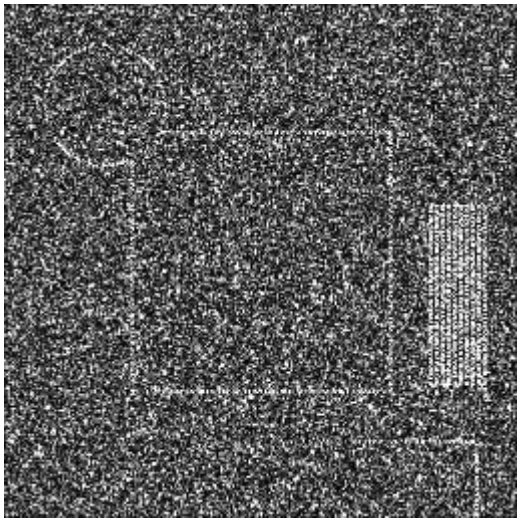


Рис. 2. Результат действия оператора Лапласа

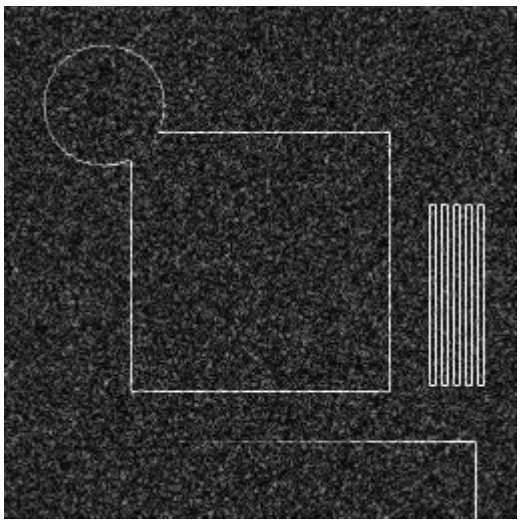


Рис. 3. Результат действия оператора Робертса

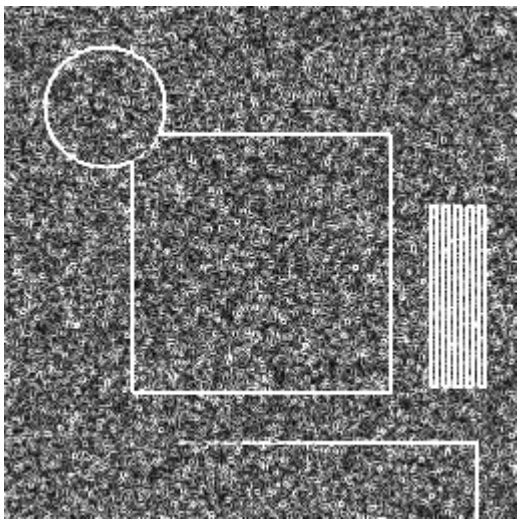


Рис. 4. Результат действия оператора Собела

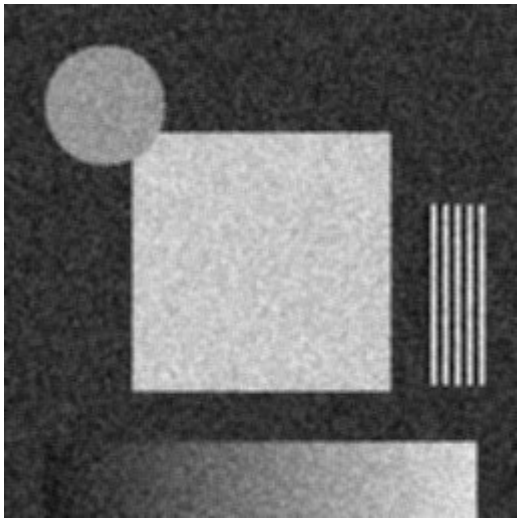


Рис. 5. Прямоугольный усредняющий фильтр 3×3

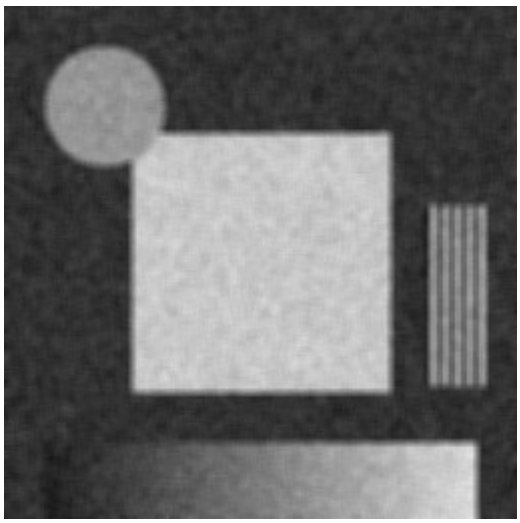


Рис. 6. Прямоугольный усредняющий фильтр 5×5

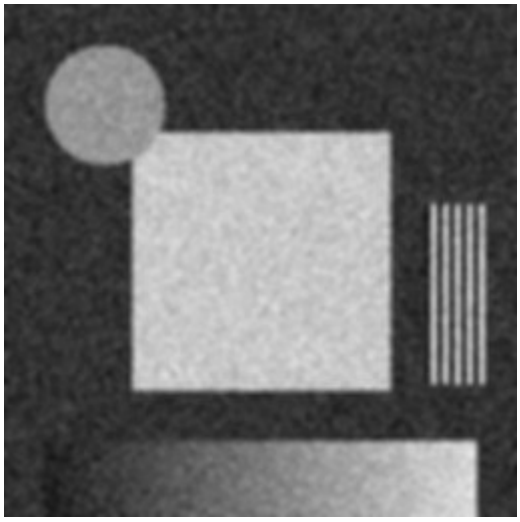


Рис. 7. Усредняющий фильтр Гаусса 5×5, σ = 1,5

5. Заключение

Выполнены первые два варианта лабораторной работы: выделение контуров операторами Лапласа, Робертса и Собела, подавление гауссова шума прямоугольным и гауссовым усреднением. Реализован класс Image с исходным интерфейсом, подготовлены программно генерируемые тестовые изображения.

В ходе сравнительного анализа выявлены следующие особенности алгоритмов:

Оператор Собела выделяет контуры с локальным поперечным сглаживанием; на шумном входе сохраняются ложные отклики, а большие амплитуды обрезаются до 255.

Оператор Робертса: обеспечивает максимальную скорость и минимальную толщину линий, но непригоден для работы в условиях шума без предварительной фильтрации.

Оператор Лапласа: чувствителен к мелким перепадам яркости и порождает двойные границы, крайне восприимчив к шуму.

Линейные фильтры уменьшают шум ценой потери резкости. На выбранном тесте окно 3×3 имеет лучшую общую MSE, а окно 5×5 — наименьшую MSE на однородном фоне. Поэтому размер окна выбирается с учётом структуры изображения.

Приложение А

Реализация функций applySobel, applyRoberts, applyLaplacian:

```
Image applySobel(const Image& src) {
    Image gray = toGrayscale(src);
    int w = gray.width();
    int h = gray.height();
    Image dst(w, h, 1);

    const int Gx[3][3] = {
       {-1, 0, 1},
       {-2, 0, 2},
       {-1, 0, 1}
    };

    const int Gy[3][3] = {
       {-1, -2, -1},
       { 0,  0,  0},
       { 1,  2,  1}
    };

    for (int y = 0; y < h; ++y) {
       for (int x = 0; x < w; ++x) {
          double sumX = 0.0;
          double sumY = 0.0;

          for (int ky = -1; ky <= 1; ++ky) {
             for (int kx = -1; kx <= 1; ++kx) {
                int pixel = getPixelChannel(gray, x + kx, y + ky, 0);
                sumX += pixel * Gx[ky + 1][kx + 1];
                sumY += pixel * Gy[ky + 1][kx + 1];
             }
          }

              int mag = clampByte(static_cast<int>(std::sqrt(sumX * sumX + sumY *
sumY)));
          dst.setColor(x, y, mag, mag, mag);
       }
    }
    return dst;
  }

  Image applyRoberts(const Image& src) {
```

```
    Image gray = toGrayscale(src);
    int w = gray.width();
    int h = gray.height();
    Image dst(w, h, 1);

    for (int y = 0; y < h; ++y) {
        for (int x = 0; x < w; ++x) {
            int p00 = getPixelChannel(gray, x,     y,     0);
            int p11 = getPixelChannel(gray, x + 1, y + 1, 0);
            int p01 = getPixelChannel(gray, x + 1, y,     0);
            int p10 = getPixelChannel(gray, x,     y + 1, 0);

            double gx = p00 - p11;
            double gy = p01 - p10;

            int mag = clampByte(static_cast<int>(std::sqrt(gx * gx + gy * gy)));
            dst.setColor(x, y, mag, mag, mag);
        }
    }
    return dst;
}

Image applyLaplacian(const Image& src) {
    Image gray = toGrayscale(src);
    int w = gray.width();
    int h = gray.height();
    Image dst(w, h, 1);

    const int kernel[3][3] = {
        { 0,  1,  0 },
        { 1, -4,  1 },
        { 0,  1,  0 }
    };

    for (int y = 0; y < h; ++y) {
        for (int x = 0; x < w; ++x) {
            int sum = 0;
            for (int ky = -1; ky <= 1; ++ky) {
                for (int kx = -1; kx <= 1; ++kx) {
                    int pixel = getPixelChannel(gray, x + kx, y + ky, 0);
                    sum += pixel * kernel[ky + 1][kx + 1];
                }
            }
```

```
            int val = clampByte(std::abs(sum));
            dst.setColor(x, y, val, val, val);
        }
    }
    return dst;
}
```

Приложение Б

Реализация функций applyBoxBlur, applyGaussianBlur:

```
Image applyBoxBlur(const Image& src, int kernelSize) {
    if (kernelSize <= 0 || kernelSize % 2 == 0 || kernelSize > 255) throw
std::invalid_argument("Odd kernel size 1..255 required");
    int w = src.width();
    int h = src.height();
    int channels = src.nChannels();
    Image dst(w, h, channels);
    int radius = kernelSize / 2;
    int count = kernelSize * kernelSize;

    for (int y = 0; y < h; ++y) {
      for (int x = 0; x < w; ++x) {
        int sumR = 0, sumG = 0, sumB = 0;

        for (int ky = -radius; ky <= radius; ++ky) {
          for (int kx = -radius; kx <= radius; ++kx) {
            int r = 0, g = 0, b = 0;
            src.getColor(clampCoord(x + kx, w), clampCoord(y + ky, h), r, g, b);
            sumR += r;
            sumG += g;
            sumB += b;
          }
        }

        dst.setColor(x, y, sumR / count, sumG / count, sumB / count);
      }
    }
    return dst;
  }

  Image applyGaussianBlur(const Image& src, int kernelSize, double sigma) {
    if (kernelSize <= 0 || kernelSize % 2 == 0 || kernelSize > 255) throw
std::invalid_argument("Odd kernel size 1..255 required");
    if (!std::isfinite(sigma) || sigma < 1e-6 || sigma > 1e6) throw
std::invalid_argument("sigma must be finite, 1e-6..1e6");
    int w = src.width();
    int h = src.height();
    int channels = src.nChannels();
    Image dst(w, h, channels);
    int radius = kernelSize / 2;
```

```
    std::vector<std::vector<double>> kernel(kernelSize,
std::vector<double>(kernelSize));
    double sumWeights = 0.0;
    double s = 2.0 * sigma * sigma;

    for (int y = -radius; y <= radius; ++y) {
      for (int x = -radius; x <= radius; ++x) {
        double r = x * x + y * y;
        double val = std::exp(-r / s) / (3.14159265358979323846 * s);
        kernel[y + radius][x + radius] = val;
        sumWeights += val;
      }
    }

    for (int i = 0; i < kernelSize; ++i) {
      for (int j = 0; j < kernelSize; ++j) {
        kernel[i][j] /= sumWeights;
      }
    }

    for (int y = 0; y < h; ++y) {
      for (int x = 0; x < w; ++x) {
        double accR = 0.0, accG = 0.0, accB = 0.0;

        for (int ky = -radius; ky <= radius; ++ky) {
          for (int kx = -radius; kx <= radius; ++kx) {
            int r = 0, g = 0, b = 0;
            src.getColor(clampCoord(x + kx, w), clampCoord(y + ky, h), r, g, b);
            double weight = kernel[ky + radius][kx + radius];
            accR += r * weight;
            accG += g * weight;
            accB += b * weight;
          }
        }

        dst.setColor(x, y, clampByte(static_cast<int>(accR + 0.5)),
                     clampByte(static_cast<int>(accG + 0.5)),
                     clampByte(static_cast<int>(accB + 0.5)));
      }
    }
    return dst;
  }
```